Foundation of Large Language Models (FLLM)

AI60213

Racing to the Sink: Why Gated Attention Still Forms Attention Sinks

Attention sinks are necessary for softmax, and preferred by training

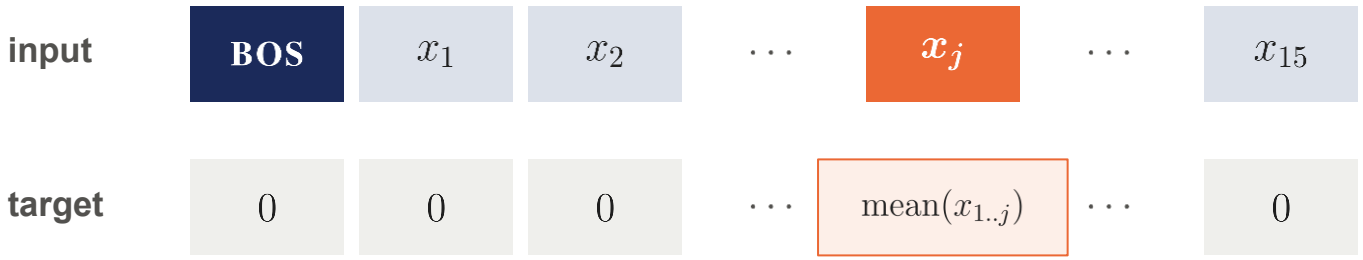
Group Number: G28

Names: Aditya Kariwal, Manish Kundu, Samyak Sheersh

Roll Numbers: [roll numbers]

Base paper: Ran-Milo (2026), Attention Sinks Are Provably Necessary in Softmax Transformers, arXiv:2603.11487

Problem Statement



the trigger sits at a random position j; every token carries 13 random content features

$$y^{(i)} = \begin{cases} \frac{1}{j}\sum_{k=1}^{j} x^{(k)} & i = j \quad \text{(trigger)} \\ 0 & i \neq j \quad \text{(no-op)} \end{cases}$$

$$\mathcal{L}(f) = \max_i \left\| y^{(i)} - f(x)^{(i)} \right\|_\infty < 5 \cdot 10^{-3}$$

Theorem (Ran-Milo, 2026)

$$\mathcal{L}(f) \leq \eta \implies \alpha_{i,\text{BOS}} \geq 1 - \varepsilon \text{ at every non-trigger row } i$$

A softmax model needs an attention sink to do nothing; ReLU attention does not.

The paper's remedy: gating or un-normalised attention.

Our question

A gated head can output zero without any sink (we construct one). Does training actually use the gate? If not, what decides which way a head switches off, and can we control it?

Data pipeline. Synthetic task: 128 fresh sequences per step, generated on the fly. Language model: TinyStories at character level, [BOS] + 255 characters per sample.

Architecture

Attention-only transformer (as in the base paper)

D layers $\times$ H heads of width d_h, residual for $D > 1$, no MLP / LayerNorm / positions

$$\text{head}(x)_i = W_O\Big(g_i \odot \textstyle\sum_{k\le i} \alpha_{ik} W_V x_k\Big), \qquad s_{ik} = q_i^\top k_k$$

$$\text{softmax:} \quad \alpha_{ik} = e^{s_{ik}} \big/ \textstyle\sum_{k'} e^{s_{ik'}}$$

$$\text{off-by-one:} \quad \alpha_{ik} = e^{s_{ik}} \big/ \big(1 + \textstyle\sum_{k'} e^{s_{ik'}}\big)$$

$$\text{gated (Qiu et al.):} \quad g_i = \sigma(W_G x_i + b)$$

Gate parameterisations we study

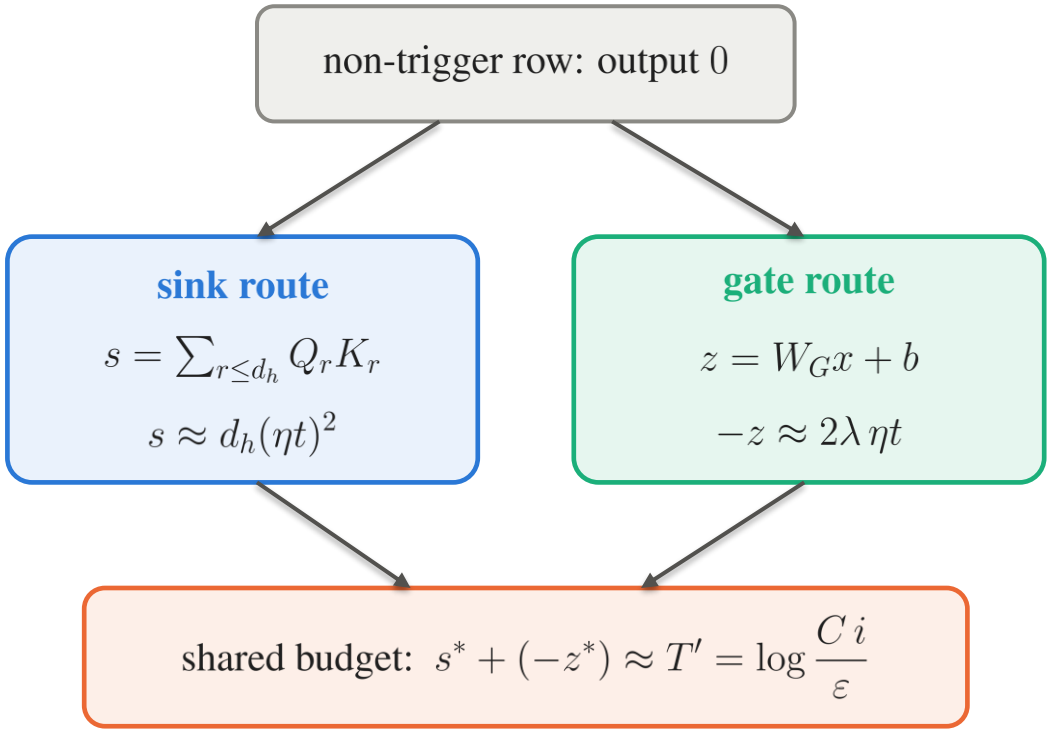
$$\text{linear} + \lambda: \quad \text{lr}(W_G, b) = \lambda \cdot \text{lr}$$

$$\text{race-matched:} \quad g_i = \sigma\big(\kappa(W_G x_i + b)\big),\ \kappa = 2.5\sqrt{d_h}$$

$$\text{bilinear:} \quad z_i = A_h(B_h x_i) + b$$

Language model: character-level GPT, 6 layers, d = 384, 6 heads of width 64, pre-LN + MLP, context 256, AdamW; same attention variants.

Two ways for a head to switch off



The faster route takes the larger share of the budget, which gives

$$\lambda^* = \frac{T' - S_0}{2\sqrt{S_0}}\sqrt{d_h}$$

Contribution Code

Written from scratch in PyTorch. Excerpts of our contributions; full code in src/.

src/sinks.py · gate parameterisations

```
if gate_kind == "bilinear":           # z = A_h (B_h x) + b
    self.W_GB = nn.Linear(d, H * dh, bias=False)
    self.W_GA = nn.Parameter(torch.empty(H, dh, dh))
else:
    self.W_G = nn.Linear(d, H * dh, bias=True)
    if gate_kind == "scaled":         # race-matched gate
        self.kappa = gate_scale * math.sqrt(dh)
self.W_G.weight.data.div_(self.kappa) # muP: init / kappa

def gate_logits(self, x):             # == gate lr x kappa
    return self.kappa * self.W_G(x)
```

src/sinks.py · racing the two routes

```
gate_p = [p for n, p in named if "W_G" in n]
qk_p = [p for n, p in named if n[:3] in ("W_Q", "W_K")]
groups = [
    dict(params=other_p, lr=lr),
    dict(params=qk_p, lr=lr * qk_lr_mult),     # sink
    dict(params=gate_p, lr=lr * gate_lr_mult), # gate
]
opt = torch.optim.Adam(groups, betas=(0.9, 0.95))
# stop when the l_inf error on a batch < 5e-3
```

src/race_theory.py · closed-form critical multiplier

```
def lam_star(dh, i=8, eps=5e-3, vbar=1.0, a0=0.6):
    T = math.log(vbar * i / eps)       # shared budget T'
    S0 = math.log(i * a0 / (1 - a0))   # sink-head logit
    return (T - S0) * math.sqrt(dh) / (2 * math.sqrt(S0))
```

src/probe.py · zero-valued gradient probes

```
p_s = torch.zeros(H, requires_grad=True)  # BOS logit
p_g = torch.zeros(H, requires_grad=True)  # gate logit
layer.probe = (row_mask, p_s, p_g)        # non-trigger rows
gs = torch.autograd.grad(loss, ps + pg)
gate_pull = g / layer.dh                  # per coordinate
```

Benchmark Methods and Evaluation

Methods compared

Method	Role	Source
softmax	baseline: sink is necessary	Ran-Milo 2026
ReLU	sink-free, un-normalised	Ran-Milo 2026
softmax-off-by-one	null slot: can switch off	Miller 2023
linear output gate	gate: can switch off	Qiu et al. 2025
gate + lr multiplier λ	speeds up the gate route	ours
race-matched gate	$\kappa = 2.5 \cdot \sqrt{d_h}$, muP-style	ours
bilinear gate	gate as fast as the sink	ours
controls	Q/K-lr ×10, all-lr ×10, SGD	ours

Evaluation metrics

Metric	Definition
BOS mass	mean attention on BOS, non-trigger rows
sink head	BOS mass > 0.6
gate value	importance-weighted, off / trigger rows
steps	until $\ell\infty$ error < $5 \cdot 10^{-3}$
s*, −z*	sink-logit gap / gate closing at stop
λ*	λ where mean BOS mass crosses 0.6
LM	val loss, BOS attention, BOS outlier

3–5

seeds per configuration

558

models trained in total

11 CPU + 2×T4

local cores and Kaggle GPUs

1 script

src/summarize.py recomputes every number

Results and Discussion

0/48 → 48/48

sink heads with gate lr ×10, then also Q/K lr ×10: the race is causal

r = -0.83

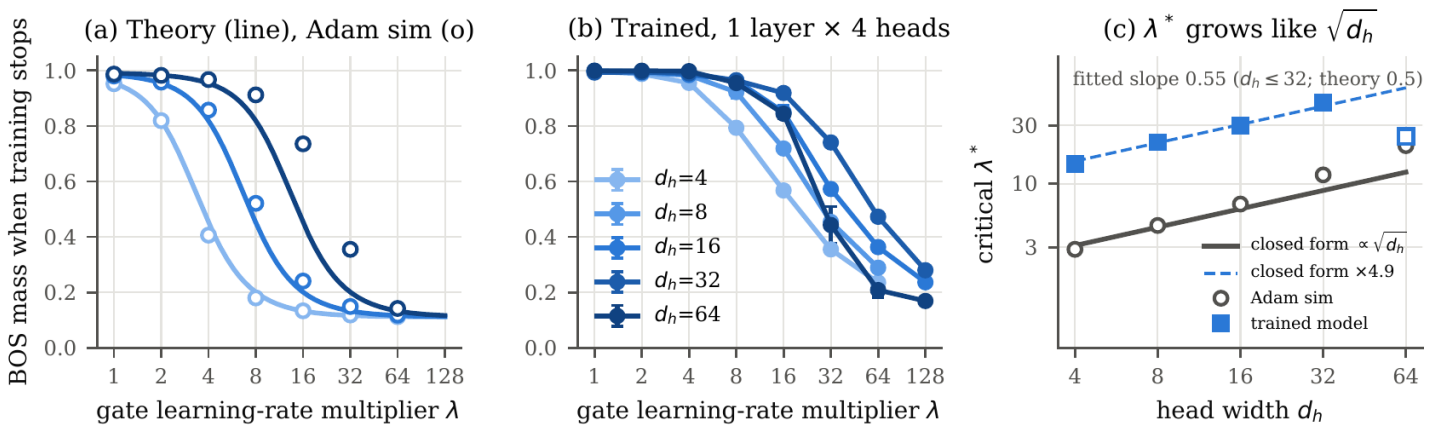
shared log-budget: s* + (−z*) = 8.8 ± 1.8

slope 0.55

λ* ∝ √d_h (theory 0.5); constant ×4.9

1.43× → 0.97×

LM: BOS activation outlier, softmax → gate λ = 10



4 layers × 4 heads, 3 seeds	sink heads	steps
baseline, λ = 1	23/48	18.4k
gate lr ×10	0/48	3.3k
gate ×10 and Q/K ×10	48/48	3.4k
all learning rates ×10	33/48	3.4k

What holds. Sinks are preferred, not only necessary: gated models keep them at every depth and gate init, though a sink-free solution exists.

What fails. SGD still forms sinks; at 4 layers λ* hardly depends on width, so the √d_h gate fails at d_h = 4.

Limitations. Synthetic task; first-order theory (constant off by ~5×); small LM in which sinks had not yet formed.

Conclusion. An off switch makes sinks unnecessary but not absent: the gate must also learn fast enough. Its learning rate decides the race.